Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación ▪ Carrera de Software

ACTIVIDAD SUMATIVA

TA-IND-01

Trabajo individual: IEEE 29148 y tipos de requisitos de un sistema conocido

Jonathan Holguín

1. Sistema elegido

GitHub Desktop para Windows permite trabajar con repositorios Git desde una interfaz gráfica. Lo usan estudiantes y desarrolladores para revisar sus cambios antes de guardarlos en un commit y compartirlos en GitHub [1].

El análisis se centra en tres acciones del trabajo con GitHub Desktop. Revisar qué cambió en los archivos, organizar esos cambios en ramas y sincronizar el repositorio local con GitHub.

2. Resumen analítico de ISO/IEC/IEEE 29148:2018

Cómo se escribe un requisito

Antes de hacer un commit, el usuario puede querer revisar un archivo para decidir si lo incluye. Esa necesidad sirve de punto de partida. Para escribir el requisito, hay que indicar qué debe mostrar el programa y cuándo [2, Sec. 5.2.3], [2, Sec. 5.2.4].

Un requisito sería **Cuando el usuario seleccione un archivo de texto modificado en la vista de cambios, GitHub Desktop debe mostrar las líneas añadidas y eliminadas desde el último commit de la rama activa.** El ejemplo indica qué hace el usuario y qué debe mostrar el programa [2, Sec. 5.2.4].

El requisito se aplica a los archivos de texto. Si el proyecto exige que el programa funcione en Windows, eso se anota como una restricción. El equipo de desarrollo puede elegir cómo calcular las diferencias [2, Sec. 5.2.4].

Un mismo archivo puede tener contenido distinto en dos ramas. Si el programa compara con el último commit de otra rama, podría mostrar cambios que el usuario no hizo en la rama que tiene abierta.

Para probarlo, se pueden preparar dos ramas con versiones distintas del mismo archivo y editar una de ellas. La vista debe mostrar los cambios frente al último commit de la rama donde se editó el archivo [2, Sec. 5.2.3], [2, Sec. 5.2.4].

Características de un requisito

El mismo ejemplo sirve para entender las nueve características que la norma pide revisar en cada requisito [2, Sec. 5.2.5].

1. **Necesario.** La función permite revisar los cambios antes del commit. Sin ella, el usuario tendría que buscar por su cuenta qué líneas cambió.

2. **Apropiado.** El nivel de detalle depende de lo que se describe. Para esta pantalla basta con decir qué líneas debe mostrar. El equipo puede decidir después cómo encontrarlas.

3. **No ambiguo.** El último commit de la rama activa indica qué versión se usa para comparar. Si dijera "la versión anterior", alguien podría entender que se refiere a la última vez que guardó el archivo.

4. **Completo.** Se entiende por sí solo porque indica qué archivo selecciona el usuario, con qué versión se compara y qué debe aparecer en pantalla.

5. **Singular.** Mostrar los cambios es una sola función. Guardarlos en un commit merece su propio requisito porque se puede probar por separado.

6. **Factible.** El equipo debe poder crear la función con el tiempo y los recursos que tiene, sin asumir un riesgo demasiado alto. Limitarla a archivos de texto ayuda a calcular cuánto trabajo requiere.

7. **Verificable.** Se añade una línea y se borra otra en un archivo de prueba. La vista de cambios debe mostrar ambas correctamente.

8. **Correcto.** Responde a lo que el usuario necesita. Para decidir qué incluir en el commit, le hace falta ver las líneas que cambiaron.

9. **Conforme.** Sigue el formato acordado. Por ejemplo, se puede usar un código para cada requisito y el verbo **debe** para señalar lo obligatorio.

El conjunto de requisitos

Un conjunto es **completo** si cubre lo que se acordó y resuelve los puntos pendientes. Es **consistente** si los requisitos no se contradicen ni se repiten y si el documento usa los mismos términos y unidades [2, Sec. 5.2.6].

También debe ser **factible**. Dos funciones pueden ser posibles por separado y aun así necesitar más tiempo o recursos de los que el equipo tiene para hacerlas juntas.

El conjunto es **comprensible** si se entiende qué debe hacer el sistema y cómo se relacionan sus partes. Se puede **validar** si es posible revisar con los usuarios si lo descrito les sirve para completar su trabajo [2, Sec. 5.2.6].

Revisar los cambios, crear un commit y enviarlo a GitHub son pasos que se pueden probar por separado. Al probarlos juntos, conviene revisar si el commit respeta la selección de archivos hecha en la pantalla anterior.

Si ese paso queda sin explicar, cada pantalla podría funcionar como se describe y aun así se podrían guardar cambios que la persona quería dejar fuera. Probar el recorrido completo ayuda a detectar ese problema [2, Sec. 5.2.5], [2, Sec. 5.2.6].

Dos requisitos pueden dar instrucciones distintas sobre el mismo paso. Uno podría pedir que el commit guarde solo los archivos marcados y otro que incluya todos los que se modificaron.

Con un solo archivo modificado y marcado, ambas instrucciones darían el mismo resultado. Con dos archivos modificados y solo uno marcado, se vería la diferencia. Ese caso ayuda a decidir si se trata de acciones distintas o si hay que corregir un requisito [2, Sec. 5.2.6].

Revisar todos los pasos también ayuda a ver qué falta. Por ejemplo, el equipo puede explicar cómo crear y enviar un commit, pero pasar por alto qué mensaje debe mostrar el programa cuando falla el envío.

En ese caso, la persona necesita saber si el commit quedó guardado en su equipo y si llegó a GitHub. Revisar el recorrido completo permite decidir si hace falta escribir un requisito para ese caso [2, Sec. 5.2.6].

Redacción y atributos

Decir que la vista de cambios debe ser "rápida" deja sin definir cuánto puede esperar el usuario. Para probarla, el equipo debe acordar cuándo empieza y termina la medición, qué archivo usará y cuánto tiempo aceptará.

La norma pide evitar frases poco claras como "la mejor interfaz", "fácil de usar", "casi siempre" o "si es posible". También conviene evitar "entre otros" si hace falta indicar exactamente qué se incluye [2, Sec. 5.2.7].

La norma también advierte sobre expresiones como "este" sin decir a qué se refiere, "más rápido" sin indicar con qué se compara y "siempre" cuando no se puede asegurar [2, Sec. 5.2.7].

Si se cita otro documento, hay que indicar su versión y la parte a la que se hace referencia [2, Sec. 5.2.7].

Con "mostrar los cambios y/o abrir el historial", alguien podría aprobar la prueba con solo ver los cambios, mientras que otra persona esperaría que también se abra el historial. Si hacen falta ambas acciones, se puede escribir un requisito para cada una e indicar cuándo debe ocurrir [2, Sec. 5.2.5], [2, Sec. 5.2.7].

Los atributos son datos que acompañan al requisito. Por ejemplo, el código RF-01 permite encontrar el requisito de mostrar los cambios aunque cambie su redacción. La justificación explica por qué se necesita [2, Sec. 5.2.8.1], [2, Sec. 5.2.8.2].

La versión indica cuál es el texto actual y la prioridad ayuda a ordenar el trabajo. El tipo también es un atributo. En este caso, el requisito es **funcional** porque describe una acción que realiza la aplicación [2, Sec. 5.2.8.2], [2, Sec. 5.2.8.3].

3. Glosario

1. **Requisito.** Enunciado que expresa una necesidad e indica las condiciones y los límites que se deben tener en cuenta [2, Sec. 3.1.19]. *Ejemplo.* Al seleccionar un archivo de texto modificado, GitHub Desktop debe mostrar las líneas que se añadieron y las que se borraron.
2. **Condición.** Circunstancia que se puede comprobar y que indica cuándo se aplica un requisito [2, Sec. 3.1.6]. *Ejemplo.* Que el archivo seleccionado tenga cambios pendientes es la condición para mostrar sus diferencias.
3. **Restricción.** Límite impuesto desde fuera del proyecto que reduce las opciones para diseñar o desarrollar el sistema [2, Sec. 3.1.7]. *Ejemplo.* Si un laboratorio solo dispone de equipos con Windows 11, la aplicación debe poder ejecutarse en esos equipos para usarse en clase.
4. **Contexto de uso.** Personas, tareas, herramientas y entorno en los que se usa el producto [2, Sec. 3.1.8]. *Ejemplo.* Un estudiante usa GitHub Desktop en una computadora del laboratorio para subir los cambios de un proyecto que comparte con su grupo de clase.
5. **Escenario operativo.** Secuencia imaginada que muestra cómo interactúan las personas, el sistema y su entorno durante una tarea [2, Sec. 3.1.17]. *Ejemplo.* Un estudiante abre su rama en GitHub Desktop, revisa el README, guarda el cambio en un commit y lo envía a GitHub con la conexión del laboratorio.
6. **Parte interesada.** Persona u organización con algún interés en el sistema o en que este responda a sus necesidades [2, Sec. 3.1.28]. *Ejemplo.* Al docente le interesa que los commits que el estudiante hace en GitHub Desktop queden en GitHub para poder revisar el avance.
7. **Usuario.** Persona o grupo que interactúa con el sistema o se beneficia de su uso [2, Sec. 3.1.35]. *Ejemplo.* Un estudiante usa GitHub Desktop para ver qué archivos cambiaron antes de elegir cuáles incluir en su commit.
8. **Trazabilidad de requisitos.** Registro de dónde viene cada requisito y cómo se reparte en requisitos más concretos o en partes del sistema [2, Sec. 3.1.23]. *Ejemplo.* Se vincula la necesidad de revisar cambios antes del commit con el requisito de mostrar las líneas añadidas y con la vista de cambios que las muestra.
9. **Verificación de requisitos.** Revisión de cada enunciado y del conjunto para comprobar que estén bien formulados [2, Sec. 3.1.26]. *Ejemplo.* Al revisar el requisito de mostrar diferencias, se comprueba si indica qué versión del archivo se toma como referencia.
10. **Validación de requisitos.** Revisión que confirma si los requisitos describen el sistema que las personas interesadas necesitan [2, Sec. 3.1.25]. *Ejemplo.* Se plantea a un estudiante si ver las líneas modificadas le permite decidir qué guardar en el commit o si también necesita elegir líneas dentro de un mismo archivo.

4. Requisitos y clasificación

La Tabla 1 reúne 15 requisitos de GitHub Desktop en Windows. Cada enunciado indica qué debe hacer la aplicación, en qué condiciones y con qué límites cuando corresponde [2, Sec. 5.2.4, Fig. 1].

Los tipos distinguen el propósito de cada requisito [2, Sec. 5.2.8.3], [3, Ch. 1, Sec. 1.4], [3, Ch. 1, Sec. 1.7].

La columna ISO/IEC 25010 muestra la característica y subcaracterística de cada requisito de calidad del producto. El guion indica que esa clasificación no aplica.

Los tiempos y límites de la tabla son metas para las pruebas. Las pruebas locales se plantean en un equipo con Windows 11 de 64 bits, SSD y 8 GB de RAM, con un repositorio de práctica.

Cuadro 1: Clasificación de requisitos de GitHub Desktop

ID	Enunciado	Tipo	ISO/IEC 25010	Métrica	Criterio de aceptación
R01	Cuando el usuario seleccione un archivo de texto modificado, GitHub Desktop debe mostrar las líneas añadidas y borradas respecto del último commit de la rama activa.	Funcional	—	Líneas visibles	Preparar dos ramas con versiones distintas del mismo archivo. En la rama activa, añadir una línea y borrar otra. La vista muestra solo esos dos cambios frente al último commit de esa rama.
R02	Cuando el usuario abra el historial, GitHub Desktop debe listar los commits de la rama activa.	Funcional	—	Commits visibles	En una rama de prueba con tres commits, el historial muestra los tres commits con sus mensajes.
R03	Si el usuario confirma un nombre válido y disponible y elige la rama activa como origen, GitHub Desktop debe crear la rama local desde el último commit de esa rama.	Funcional	—	ID del commit inicial	Al terminar, las dos ramas apuntan al mismo commit.
R04	Cuando el usuario guarde los cambios de un archivo con *Stash*, GitHub Desktop debe mostrar una entrada con esos cambios en la misma rama.	Funcional	—	Entradas visibles	Al guardar un cambio del README, se puede abrir la entrada y ver el cambio.
R05	Al crear un commit desde la vista de cambios, GitHub Desktop debe limitar las adiciones del commit a las líneas que el usuario marcó.	Funcional	—	Líneas incluidas	Añadir dos líneas en bloques separados y desmarcar una. El commit incluye solo la línea que quedó marcada.
R06	Cuando el usuario elija otra rama local en un repositorio sin cambios pendientes, GitHub Desktop debe activar la rama elegida.	Funcional	—	Rama activa	Al pasar de `main` a `prueba`, la aplicación indica `prueba` como rama activa.
R07	Cuando el usuario elija abrir el repositorio en el editor, GitHub Desktop debe pasar la ruta de la carpeta principal del repositorio al editor configurado.	Interfaz externa	—	Carpeta abierta	Con Visual Studio Code como editor, se abre la carpeta principal del repositorio activo.
R08	Cuando el usuario pulse *Push origin* con conexión y permiso de escritura, GitHub Desktop debe enviar el commit pendiente a la rama de GitHub vinculada a la rama activa.	Interfaz externa	—	ID del commit en GitHub	Tras enviar un commit, el último commit de la rama en GitHub tiene el mismo ID que el commit local.
R09	GitHub Desktop debe abrir repositorios locales en los equipos de práctica con Windows 11 de 64 bits.	Restricción	—	Apertura del repositorio	En el equipo de prueba, la vista de cambios carga el repositorio local elegido.

Tabla 1. Clasificación de requisitos (continuación)

ID	Enunciado	Tipo	ISO/IEC 25010	Métrica	Criterio de aceptación
R10	GitHub Desktop debe ofrecer un flujo de commit local en el que, en su primer uso, al menos cuatro de cinco estudiantes que ya saben crear un commit desde la terminal guarden el archivo preparado en un commit en tres minutos o menos y sin ayuda.	Calidad en uso	—	Estudiantes que completan la tarea	Antes de la prueba, cada estudiante crea un commit desde la terminal. En GitHub Desktop, medir desde que se abre la vista de cambios hasta que aparece el commit en el historial. Cumplen al menos cuatro de cinco en tres minutos o menos y sin ayuda.
R11	Cuando el usuario seleccione un archivo de texto de 1 000 líneas con 20 líneas modificadas, GitHub Desktop debe mostrar sus diferencias en 2 s o menos.	Calidad	Eficiencia de desempeño / Comportamiento temporal [4, Sec. 3.2.1]	Tiempo de respuesta (s)	Medir desde el clic hasta que aparecen las diferencias. Cada uno de los 10 intentos tarda como máximo 2 s.
R12	Durante la actualización de la vista de cambios con 200 archivos de texto modificados de 10 KiB cada uno, GitHub Desktop debe usar hasta 250 MiB más de memoria privada que antes de actualizar.	Calidad	Eficiencia de desempeño / Utilización de recursos [4, Sec. 3.2.2]	Aumento máximo de memoria (MiB)	Sumar la memoria privada de los procesos de la aplicación. En tres pruebas, restar al máximo el valor inicial. La diferencia es de 250 MiB o menos en cada prueba.
R13	Con la opción de ignorar espacios desactivada, GitHub Desktop debe mostrar las mismas líneas añadidas y borradas que `git diff HEAD` para cada archivo de texto modificado.	Calidad	Adecuación funcional / Corrección funcional [4, Sec. 3.1.2]	Archivos con diferencias correctas	En 10 archivos registrados en Git, deben coincidir la cantidad, el texto y la marca de añadido o borrado de cada línea.
R14	GitHub Desktop debe mostrar los mismos commits que `git log` al consultar la misma rama activa del mismo repositorio local.	Calidad	Compatibilidad / Interoperabilidad [4, Sec. 3.3.2]	Commits visibles en ambas herramientas	Crear dos commits con cada herramienta. Los cuatro aparecen con los mismos ID en ambos historiales.
R15	Si el aviso de descarte está activado, GitHub Desktop debe pedir confirmación antes de borrar los cambios de un archivo.	Calidad	Capacidad de interacción / Protección frente a errores del usuario [4, Sec. 3.4.4]	Descartes detenidos al cancelar	En los cinco intentos aparece el aviso. Al cancelar, se conserva el cambio en cada archivo.

Capturas de GitHub Desktop

Las capturas se tomaron en un repositorio local de prueba. La Figura 1 muestra las líneas modificadas en la rama `main` (R01), la Figura 2 muestra sus tres commits (R02) y la Figura 3 muestra el aviso antes de descartar cambios del README (R15).

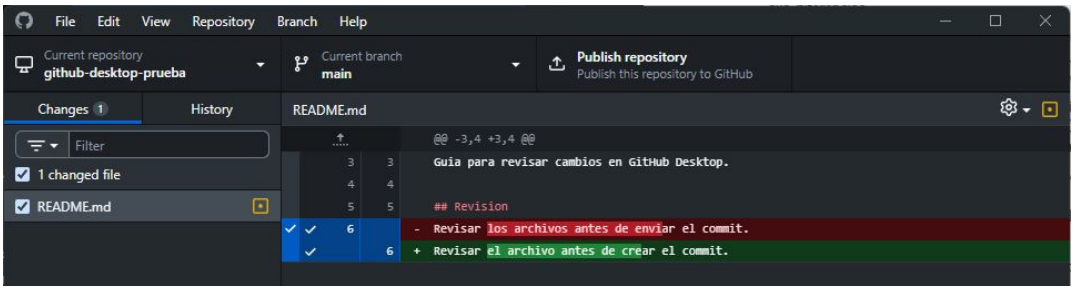


Figura 1: R01. Una línea eliminada y una añadida en la rama `main`.

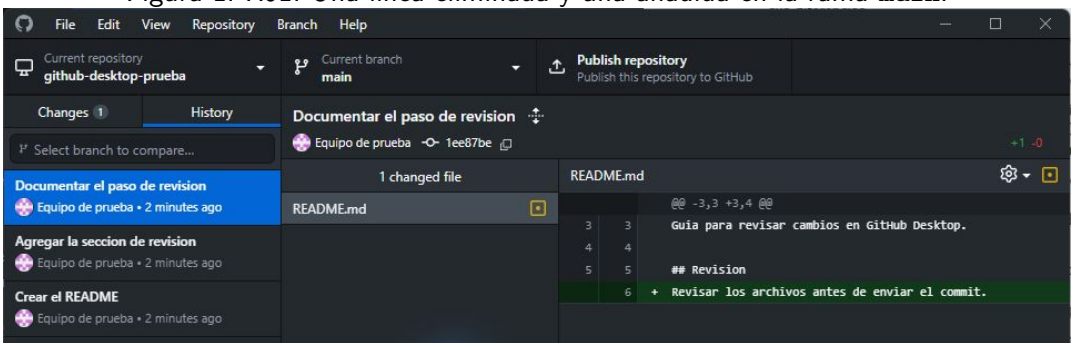


Figura 2: R02. Historial con los tres commits de la rama `main`.

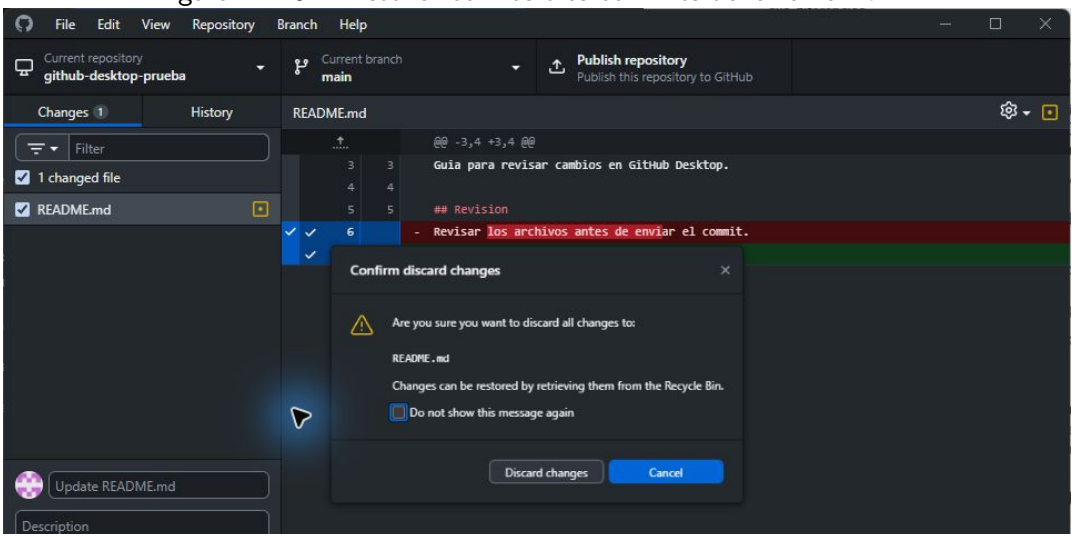


Figura 3: R15. Confirmación antes de descartar los cambios del README.

5. Autoevaluación de los requisitos

6. Reflexión

7. Uso de IA

Referencias

[1] GitHub. “About GitHub Desktop”. GitHub Docs. Accessed: Nov. 11, 2026. [Online]. Available: https://docs.github.com/en/desktop/overview/about-github-desktop

[2] *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE Std. 29148:2018, Nov. 2018, doi: 10.1109/IEEESTD.2018.8559686.

[3] H. Washizaki, Ed., *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide)*. IEEE Computer Society, Sep. 2025, version 4.0a. [Online]. Available: https://ieeecs-media.computer.org/media/education/swebok/swebok-v4.pdf

[4] *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model*, ISO/IEC Std. 25010:2023, Nov. 2023. [Online]. Available: https://www.iso.org/standard/78176.html